

FIAP – Pós-graduação em IA para Devs 6IADT

Tech Challenge Fase 1

Alunos

Luís Gustavo de Araújo Silva - RM 366233

Vinicius Santos de Oliveira - RM 366276

Repositório

<https://github.com/Luis-Silva80/tech-challenge>

Código fonte

<https://github.com/Luis-Silva80/tech-challenge/blob/develop/main.py>

Etapa 1: Importação das bibliotecas usadas no modelo

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sb
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, confusion_matrix,  
classification_report
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.model_selection import cross_val_score
```

Etapa 2: Importando a base

```
diabetes_df = pd.read_csv("./datasets/diabetes.csv")
```

```
# Etapa 3: Visualizar os primeiros 10 registros e estatísticas do Dataset

print('Etapa 3: Visualizar os primeiros registros e estatísticas do Dataset\n')

print(f'Primeiros dados: \n{diabetes_df.head()}\n')

print(f'Estatísticas: \n{diabetes_df.describe()}\n')
```

```
# Etapa 4: Verificar se existem valores duplicados

print('Etapa 4: Verificar se existem valores duplicados\n')

print(f'Valores duplicados: {diabetes_df.duplicated().sum()}\n')

no_duplicates_df = diabetes_df.drop_duplicates()
```

```
# Etapa 5: Verificar se existem valores nulos

print('Etapa 5: Verificar se existem valores nulos\n')

print(f'Valores nulos: \n{no_duplicates_df.isnull().sum()}\n')

no_nulls_df = no_duplicates_df.dropna()
```

```
# Etapa 6: Padronizar todos os dados como float

print('Etapa 6: Padronizar todos os dados como float\n')

diabetes_df = diabetes_df.astype(float)

print(f'Dados como tipo float: \n{diabetes_df.head()}\n')
```

```
# Etapa 7: Análise de dados de acordo gravidez e idade

print('Etapa 7: Análise de dados de acordo gravidez e idade\n')

preg_min = diabetes_df['Pregnancies'].min()

preg_max = diabetes_df['Pregnancies'].max()

print(f'Menor parteira: \n{diabetes_df[diabetes_df['Pregnancies'] ==  
preg_min].iloc[0]}\n')
```

```
print(f'Maior parteira: \n{diabetes_df[diabetes_df["Pregnancies"] ==  
preg_max].iloc[0]}\n')
```

```
age_min = diabetes_df['Age'].min()
```

```
age_max = diabetes_df['Age'].max()
```

```
print(f'Menor idade: \n{diabetes_df[diabetes_df["Age"] == age_min].iloc[0]}\n')
```

```
print(f'Maior idade: \n{diabetes_df[diabetes_df["Age"] == age_max].iloc[0]}\n')
```

```
# Etapa 8: Analisando correlação dos dados
```

```
print('Etapa 8: Analisando correlação dos dados\n')
```

```
correlation_matrix = diabetes_df.select_dtypes(include=["float64",  
'int']).corr().round(2)
```

```
fig, ax = plt.subplots(figsize=(8, 8))
```

```
sb.heatmap(data=correlation_matrix, annot=True, linewidths=.5, ax=ax)
```

```
plt.show()
```

```
# Etapa 9: Análise do gráfico de dispersão dos casos de diabetes de acordo  
com a Idade
```

```
print('Etapa 9: Análise do gráfico de dispersão dos casos de diabetes de acordo  
com a Idade\n')
```

```
plt.xlabel('Age')
```

```
plt.ylabel('Outcome')
```

```
plt.scatter(diabetes_df['Age'], diabetes_df['Outcome'])
```

```
plt.show()
```

```
# Etapa 10: Análise do histograma da distribuição de idade
```

```
print('Etapa 10: Análise do histograma da distribuição de idade\n')
```

```
plt.xlabel('Age')
```

```
plt.hist(diabetes_df['Age'], bins=10, edgecolor='black', alpha=0.7)
```

```
plt.show()
```

Etapa 11: Análise Boxplot do Target com as colunas

```
colunas = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',  
'BMI', 'DiabetesPedigreeFunction']
```

11.1: Criar figura e eixos

```
fig, axes = plt.subplots(3, 3, figsize=(15, 10)) # 3 linhas x 3 colunas
```

```
axes = axes.flatten() # Facilita indexar
```

11.2: Loop para criar cada boxplot

```
for i, coluna in enumerate(colunas):
```

```
    sb.boxplot(x='Outcome', y=coluna, data=diabetes_df, ax=axes[i])
```

```
    axes[i].set_xlabel('Outcome')
```

```
    axes[i].set_ylabel(coluna)
```

11.3: Se sobrarem subplots vazios, remover

```
for j in range(len(colunas), len(axes)):
```

```
    fig.delaxes(axes[j])
```

Etapa 12: Treinamento do Modelo com o Dataframe

```
print('Etapa 12: Treinamento do Modelo com o Dataframe\n')
```

```
y = diabetes_df['Outcome']
```

```
x = diabetes_df.drop('Outcome', axis=1)
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,  
random_state=42, stratify=y)
```

Etapa: 13 Análise com KNN

```
print('Etapa: 13 Análise com KNN\n')
```

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
print(f'Shape da base de treino X: {x_train.shape}')
```

```
print(f'Shape da base teste Y: {y_test.shape}\n')
```

```
# 13.1: Treinando o modelo
```

```
knn_model.fit(x_train, y_train)
```

```
# 13.2: Previsão do primeiro dado do dataset
```

```
predict = knn_model.predict(x_test)
```

```
# 13.3: Análise da previsão
```

```
print(f'Acurácia KNN: {accuracy_score(y_test, predict)}')
```

```
print(f'Matriz de confusão KNN:\n{confusion_matrix(y_test, predict)}\n')
```

```
print(f'Classification Report: \n{classification_report(y_test, predict)}\n')
```

```
# 13.4: Calculando os erros de valores K entre 1 e 10
```

```
error = []
```

```
for i in range(1, 10):
```

```
    knn = KNeighborsClassifier(n_neighbors=i)
```

```
    knn.fit(x_train, y_train)
```

```
    pred_i = knn.predict(x_test)
```

```
    error.append(np.mean(pred_i != y_test))
```

```
# 13.5: Exibição das métricas para avaliação
```

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(range(1, 10), error, color='red', linestyle='dashed', marker='o',  
markerfacecolor='blue', markersize=10)
```

```
plt.title('Error Rate K Value')
```

```
plt.xlabel('K Value')
```

```
plt.ylabel('Mean Error')
```

```
plt.show()
```

```
# Etapa 14: Análise com Decision Tree
```

```
print('Etapa 14: Análise com Decision Tree')
```

```
decision_tree_model = DecisionTreeClassifier(random_state=42)
```

```
# 14.1: Treinando o modelo
```

```
decision_tree_model.fit(x_train, y_train)
```

```
predict = decision_tree_model.predict(x_test)
```

```
# 14.2: Acurácia com o modelo de Decision Tree
```

```
print(f'Acurácia Decision Tree: {accuracy_score(y_test, predict)}\n')
```

```
print(f'Matriz de confusão Decision Tree:\n{confusion_matrix(y_test, predict)}\n')
```

```
print(f'Classification Report Decision Tree:\n{classification_report(y_test, predict)}\n')
```

```
# Etapa 15: Análise com Random Forest
```

```
print('Etapa 15: Análise com Random Forest')
```

```
random_forest_model = RandomForestClassifier(random_state=42)
```

```
# 15.1: Treinando o modelo
```

```
random_forest_model.fit(x_train, y_train)
```

```
predict_random_forest = random_forest_model.predict(x_test)
```

```
# 15.2: Acurácia com o modelo de Random Forest
```

```
print(f'\nAcurácia Random Forest: {accuracy_score(y_test, predict_random_forest)}\n')
```

```
print(f'Matriz de confusão:\n{confusion_matrix(y_test, predict_random_forest)}\n')
```

```
print(f'Classification Report RandomForest:\n{classification_report(y_test,
predict_random_forest)}')
```

Etapa 16: Identificar colunas com valores inválidos

```
print('Etapa 16: Identificar colunas com valores inválidos\n')
```

```
invalid_columns = [
```

```
    'Glucose',
```

```
    'BloodPressure',
```

```
    'SkinThickness',
```

```
    'Insulin',
```

```
    'BMI'
```

```
]
```

16.1: Colunas com valores inválidos

```
print("\nColunas com valores inválidos')
```

```
for col in invalid_columns:
```

```
    zeros = (diabetes_df[col] == 0).sum()
```

```
    total = diabetes_df.shape[0]
```

```
    print(f'{col}: {zeros} valores inválidos ({(zeros / total) * 100:.2f}%)')
```

16.2: Colunas com valores inválidos após substituir pela média

```
print("\nColunas com valores inválidos após substituir pela média')
```

```
for col in invalid_columns:
```

```
    mean_value = diabetes_df[diabetes_df[col] != 0][col].mean()
```

```
    diabetes_df[col] = diabetes_df[col].replace(0, mean_value)
```

```
    zeros = (diabetes_df[col] == 0).sum()
```

```
    total = diabetes_df.shape[0]
```

```
    print(f'{col}: {zeros} valores inválidos ({(zeros / total) * 100:.2f}%)')
```

```
# Contagem da Target
```

```
print(f'\nContagem da Target: \n{diabetes_df['Outcome'].value_counts()}\n')
```

```
# Etapa 17: Undersample do Dataset
```

```
print('\nEtapa 17: Undersample do Dataset\n')
```

```
class_no_diabetes = diabetes_df[diabetes_df['Outcome'] == 0]
```

```
class_has_diabetes = diabetes_df[diabetes_df['Outcome'] == 1]
```

```
# 17.1: Undersample da classe 0 para ter o mesmo número de registros da classe 1
```

```
class_no_diabetes_under =
```

```
class_no_diabetes.sample(len(class_has_diabetes), random_state=42)
```

```
# 17.2: Juntar as duas classes balanceadas
```

```
diabetes_df_undersampled = pd.concat([class_no_diabetes_under,  
class_has_diabetes])
```

```
# 17.3: Embaralhar os dados dentro dataset
```

```
diabetes_df_undersampled = diabetes_df_undersampled.sample(  
    frac=1,  
    random_state=42  
)  
.reset_index(drop=True)
```

```
# 17.4: Análise dos dados com o tratamento
```

```
print(f'Análise do Dataframe Undersampled:  
\n{diabetes_df_undersampled.describe()}')
```

```
print(f'Contagem da Target:  
\n{diabetes_df_undersampled['Outcome'].value_counts()}\n')
```

```
# 17.5: Treinamento do Modelo Undersampled
```

```
y = diabetes_df_undersampled['Outcome']
```



```
x = diabetes_df_undersampled.drop('Outcome', axis=1)
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# Etapa: 18 Análise com KNN
```

```
print('Etapa: 18 Análise com KNN')
```

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
print(f'Shape da base de treino X: {x_train.shape}')
```

```
print(f'Shape da base teste Y: {y_test.shape}\n')
```

```
# 18.1: Treinando o modelo
```

```
knn_model.fit(x_train, y_train)
```

```
# 18.2: Previsão do primeiro dado do dataset
```

```
predict = knn_model.predict(x_test)
```

```
# 18.3: Análise da previsão
```

```
print(f'Acurácia KNN: {accuracy_score(y_test, predict)}\n')
```

```
print(f'Matriz de confusão KNN:\n{confusion_matrix(y_test, predict)}\n')
```

```
print(f'Relatório de Classificação: \n{classification_report(y_test, predict)}\n')
```

```
# 18.4: Calculando os erros de valores K entre 1 e 10
```

```
error = []
```

```
for i in range(1, 10):
```

```
knn = KNeighborsClassifier(n_neighbors=i)
knn.fit(x_train, y_train)
pred_i = knn.predict(x_test)
error.append(np.mean(pred_i != y_test))
```

18.5: Exibição das métricas para avaliação

```
plt.figure(figsize=(12, 6))
plt.plot(range(1, 10), error, color='red', linestyle='dashed', marker='o',
markerfacecolor='blue', markersize=10)
plt.title('Error Rate K Value')
plt.xlabel('K Value')
plt.ylabel('Mean Error')
plt.show()
```

Etapa 19: Análise com Decision Tree

```
print('Etapa 19: Análise com Decision Tree')
decision_tree_model = DecisionTreeClassifier(random_state=42)
```

19.1: Treinando o modelo

```
decision_tree_model.fit(x_train, y_train)
predict = decision_tree_model.predict(x_test)
```

19.2: Acurácia com o modelo de Decision Tree

```
print(f'Acurácia Decision Tree: {accuracy_score(y_test, predict)}\n')
print(f'Matriz de confusão Decision Tree:\n{confusion_matrix(y_test, predict)}\n')
print(f'Relatório de Classificação:\n{classification_report(y_test, predict)}\n')
```

Etapa 20: Análise com Random Forest

```
print('Etapa 20: Análise com Random Forest')
```

```
random_forest_model = RandomForestClassifier(random_state=42)
```

```
# 20.1: Treinando o modelo
```

```
random_forest_model.fit(x_train, y_train)
```

```
predict_random_forest = random_forest_model.predict(x_test)
```

```
# 20.2: Acurácia com o modelo de Random Forest
```

```
print(f'\nAcurácia Random Forest: {accuracy_score(y_test,  
predict_random_forest)}\n')
```

```
print(f'Matriz de confusão:\n{confusion_matrix(y_test,  
predict_random_forest)}\n')
```

```
print(f'Relatório de Classificação:\n{classification_report(y_test,  
predict_random_forest)}')
```

```
# Etapa 21: Oversample do Dataset
```

```
print('Etapa 21: Oversample do Dataset\n')
```

```
# 21.1: Oversample da classe 1 para ter o mesmo numero de registros da  
classe 0
```

```
class_has_diabetes_over = class_has_diabetes.sample(  
    len(class_no_diabetes),  
    random_state=42,  
    replace=True  
)
```

```
# 21.2: Juntar as duas classes balanceadas
```

```
diabetes_df_oversampled = pd.concat([class_no_diabetes,  
class_has_diabetes_over], axis=0)
```

```
# 21.3: Embaralhar os dados dentro dataset
```

```
diabetes_df_oversampled = diabetes_df_oversampled.sample(
```

```
    frac=1,  
    random_state=42  
)reset_index(drop=True)
```

21.4: Análise dos dados com o tratamento

```
print(f'Análise do Dataframe Oversampled:  
\n{diabetes_df_oversampled.describe()}\n')  
  
print(f'Contagem da Target:  
\n{diabetes_df_oversampled['Outcome'].value_counts()}\n')
```

21.5: Treinamento do Modelo Oversampled

```
y = diabetes_df_oversampled['Outcome']  
x = diabetes_df_oversampled.drop('Outcome', axis=1)  
x_train, x_test, y_train, y_test = train_test_split(  
    x,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

Etapa 22: Análise com KNN

```
print('Etapa: 22 Análise com KNN')  
  
knn_model = KNeighborsClassifier(n_neighbors=5)  
print(f'Shape da base de treino X: {x_train.shape}')  
print(f'Shape da base teste Y: {y_test.shape}\n')
```

22.1: Treinando o modelo

```
knn_model.fit(x_train, y_train)
```

22.2: Previsão do primeiro dado do dataset

```
predict = knn_model.predict(x_test)
```

22.3: Análise da previsão

```
print(f'Acurácia KNN: {accuracy_score(y_test, predict)}\n')
```

```
print(f'Matriz de confusão KNN:\n{confusion_matrix(y_test, predict)}\n')
```

```
print(f'Relatório de Classificação: \n{classification_report(y_test, predict)}\n')
```

22.4: Calculando os erros de valores K entre 1 e 10

```
error = []
```

```
for i in range(1, 10):
```

```
    knn = KNeighborsClassifier(n_neighbors=i)
```

```
    knn.fit(x_train, y_train)
```

```
    pred_i = knn.predict(x_test)
```

```
    error.append(np.mean(pred_i != y_test))
```

22.5: Exibição das métricas para avaliação

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(range(1, 10), error, color='red', linestyle='dashed', marker='o',  
markerfacecolor='blue', markersize=10)
```

```
plt.title('Error Rate K Value')
```

```
plt.xlabel('K Value')
```

```
plt.ylabel('Mean Error')
```

```
plt.show()
```

Etapa 23: Análise com Decision Tree

```
print('Etapa 23: Análise com Decision Tree')
```

```
decision_tree_model = DecisionTreeClassifier(random_state=42)
```

23.1: Treinando o modelo

```
decision_tree_model.fit(x_train, y_train)
predict = decision_tree_model.predict(x_test)
```

23.2: Acurácia com o modelo de Decision Tree

```
print(f'Acurácia Decision Tree: {accuracy_score(y_test, predict)}\n')
print(f'Matriz de confusão Decision Tree:\n{confusion_matrix(y_test, predict)}\n')
print(f'Relatório de Classificação:\n{classification_report(y_test, predict)}\n')
```

Etapa 24: Análise com Random Forest

```
print('Etapa 24: Análise com Random Forest')
random_forest_model = RandomForestClassifier(random_state=42)
```

24.1: Treinando o modelo

```
random_forest_model.fit(x_train, y_train)
predict_random_forest = random_forest_model.predict(x_test)
```

24.2: Acurácia com o modelo de Random Forest

```
print(f'\nAcurácia Random Forest: {accuracy_score(y_test,
predict_random_forest)}\n')
print(f'Matriz de confusão:\n{confusion_matrix(y_test,
predict_random_forest)}\n')
print(f'Relatório de Classificação:\n{classification_report(y_test,
predict_random_forest)}')
```

Etapa 25: Testando validação cruzada

```
print('Etapa 25: Testando validação cruzada\n')
scores = cross_val_score(random_forest_model, x, y, cv=10,
scoring='accuracy')
```

```
# 25.1: Scores da validação cruzada
```

```
print("Scores da validação cruzada (10 folds):")
```

```
print(scores)
```

```
# 25.2: Média da validação
```

```
print(f'Média da validação: {scores.mean()}')
```

README.md

<https://github.com/Luis-Silva80/tech-challenge/blob/develop/README.md>

Diabetes Dataset

<https://www.kaggle.com/datasets/mathchi/diabetes-data-set/data>

Relatório Técnico

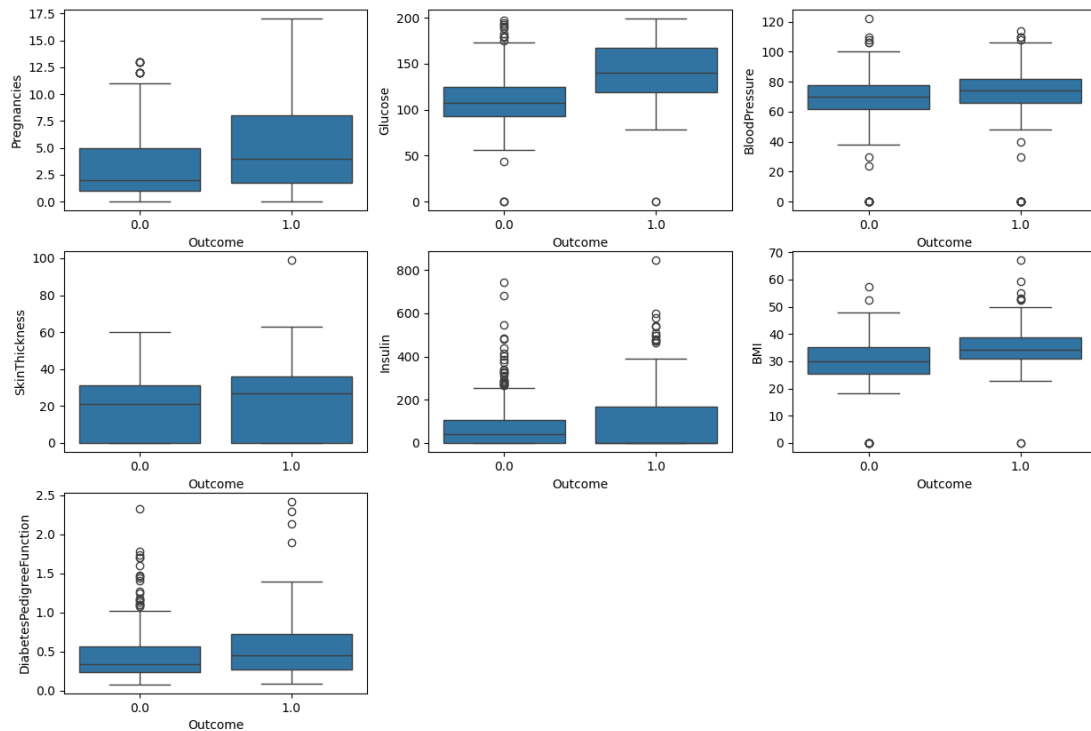
1. Estratégias de pré-processamento

Para esta primeira etapa do processo de treinamento dos modelos foi realizado o processo de Data Cleaning, para garantir que teríamos uma consistência maior nos dados, em nosso caso não havia valores nulos nem duplicados no dataset.

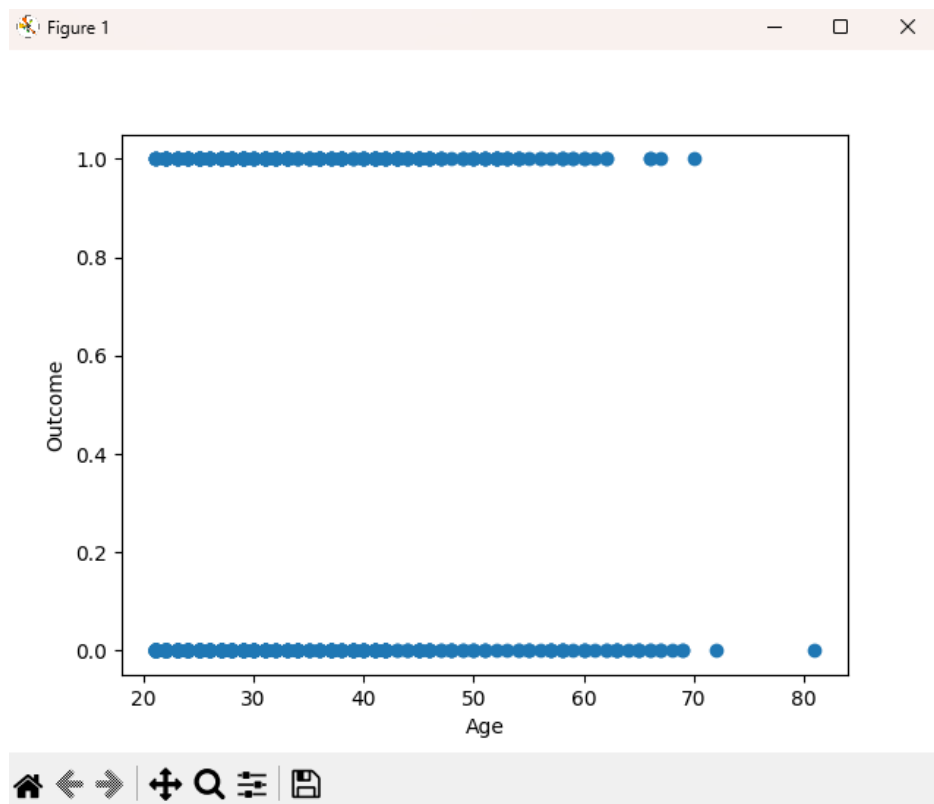
Para a normalização dos dados verificamos que as Colunas “BMI” e “DiabetesPedigreeFunction” do dataset se encontravam como float e decidimos padronizar todas as outras a partir dela para garantir que todas as colunas tivessem os mesmos critérios e assim ajudar os modelos escolhidos a lidarem melhor com os dados.

Dados como tipo float:									
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6.0	148.0	72.0	35.0	0.0	33.6	0.627	50.0	1.0
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351	31.0	0.0
2	8.0	183.0	64.0	0.0	0.0	23.3	0.672	32.0	1.0
3	1.0	89.0	66.0	23.0	94.0	28.1	0.167	21.0	0.0
4	0.0	137.0	40.0	35.0	168.0	43.1	2.288	33.0	1.0

Para a identificação dos outliers utilizamos o método Boxplot que determina a concentração dos dados para cada coluna em relação aos outliers, a partir disso decidimos não os remover. Após as validações dos modelos identificamos que os outliers não tiveram um grande impacto nas métricas de validação e nossa equipe concordou em mantê-los a fim de garantir uma maior diversidade de dados para os modelos.



Inicialmente consideramos criar colunas interpolando as colunas existentes, porém esse método não surtiu efeito prático do ponto de vista de correlação com a variável target então descartamos o método, outro ponto determinante para a estratégia apresentada foi a falta de correlações fortes das features com a target ou mesmo entre si.



2. Modelos usados

Dado o contexto do dataset e a variável target ser binária, o que claramente define uma disposição de categoria nos dados, escolhemos usar modelos de classificação dos mais simples ao mais robusto.

Nesse caso escolhemos KNeighborsClassifier, DecisionTreeClassifier e RandomForestClassifier, porém após as métricas de validação como acurácia e

matriz de confusão determinamos que RandomForestClassifier se saiu melhor em todos os critérios. Como esse modelo se utiliza de diversas árvores de decisão durante o treinamento ele tem uma capacidade maior de generalizar os dados. Como escolhemos não remover os outliers esse modelo é mais recomendado devido a sua não hipersensibilidade a outliers.

3. Resultados e Interpretação dos Dados

Como métricas de validação dos modelos utilizamos a Acurácia, Matriz de Confusão, Precisão, Recall e F1-Score.

	precision	recall	f1-score	support
0.0	0.95	0.83	0.89	100
1.0	0.85	0.96	0.90	100
accuracy			0.90	200
macro avg	0.90	0.90	0.89	200
weighted avg	0.90	0.90	0.89	200

Essas técnicas unidas ao contexto de nossos dados nos levaram a concluir que temos um modelo bom nos principais critérios, como por exemplo, acurácia geral de 90% e recall de 96% em casos de diagnóstico positivo. Porém um recall de 83% para determinar casos com diagnóstico negativo é um ponto de atenção importante, pois um falso negativo pode ser mais perigoso por se tratar de vidas humanas e nenhum erro é aceitável nesse contexto. Apesar disso, um recall de 83% pode sim ser benéfico para apoiar os médicos em diagnósticos, mas sempre levando em consideração que o médico com toda a sua vasta gama de conhecimento e anos de estudos, devem obrigatoriamente ter a palavra final e utilizar esse modelo como uma primeira fonte de interpretação dos dados do paciente, mas não como definidor final.